

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS:

Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

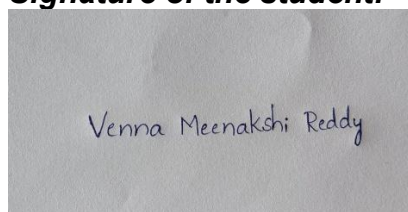
1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

/ VENNA MEENAKSHI REDDY (192471048) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a piece of paper with a handwritten signature in blue ink. The signature reads "Venna Meenakshi Reddy".

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

QUESTION 1:

1: HTTP Response Analysis

A web server returns the following HTTP response header:
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

Questions:

- 1. Identify any incorrect or invalid fields in the header.
- 2. Analyze the issue with Content-Length and explain its impact.
- 3. Determine whether the status code matches the response correctness.
- 4. Suggest corrections for the identified errors.

ANSWER:

1. Identify any incorrect or invalid fields in the header.

The HTTP response contains the following issue:

Header Field	Status	Analysis
HTTP/1.1 200 OK	Valid	Indicates that the server successfully processed the request.
Date	Valid Format	Specifies the date and time when the response was generated.
Content-Type: text/html	Valid	Correctly identifies the response body as an HTML document.
Content-Length: -150	Invalid	The value cannot be negative. According to the HTTP specification, Content-Length must be a non-negative decimal integer representing the number of bytes in the message body.

Incorrect Field:

- Content-Length: -150

2. Analyze the issue with Content-Length and explain its impact.

The **Content-Length** header specifies the exact size of the response body in **bytes**. Its value is used by the client (browser or application) to determine when the response has been completely received.

Problem

The given value is:

Content-Length: -150

This is invalid because:

- HTTP requires Content-Length to be 0 or a positive integer.
- A negative value has no meaning because a message body cannot contain a negative number of bytes.

Impact

A negative Content-Length may cause several problems:

- The client may reject the response as malformed.
- Browsers may display an error or fail to load the webpage.
- Some HTTP libraries terminate the connection immediately due to protocol violations.
- Intermediaries such as proxies or caches may ignore or discard the response.
- Persistent connections (keep-alive) can become unreliable because the client cannot determine where the response body ends.

Thus, an invalid Content-Length affects both data integrity and communication reliability.

3. Determine whether the status code matches the response correctness.

The status code is:

200 OK

A 200 OK status indicates that:

- The server successfully processed the client's request.
- The response is complete and correctly formatted.

However, the response header contains an invalid Content-Length value, making the HTTP response malformed.

Therefore:

- The 200 OK status does not fully match the correctness of the response.
- Although the requested resource may exist, the invalid header prevents the response from complying with the HTTP protocol.

Hence, the response should not be considered completely correct, even though the status code indicates success.

4. Suggest corrections for the identified errors.

The Content-Length value should be replaced with the actual size of the HTML document in bytes.

Corrected Example

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

If the HTML document contains 150 bytes, then:

Content-Length: 150

is valid.

Alternatively, if the exact size is unknown during transmission, the server may omit the Content-Length header and use Transfer-Encoding: chunked (supported in HTTP/1.1).

Conclusion

The HTTP response is mostly valid, but the Content-Length header contains a protocol violation because it specifies a negative value (-150). This prevents clients from correctly determining the message body size and may cause browsers, proxies, or applications to reject the response. Although the status code 200 OK indicates successful processing of the request, the malformed header makes the response inconsistent with HTTP standards. Replacing the invalid value with the correct positive byte count (e.g., 150) or using Transfer-Encoding: chunked ensures compliance with the HTTP/1.1 specification and reliable communication between the server and client.

QUESTION 2:

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Questions:

- 1. Identify missing or improperly closed elements.
- 2. Analyze how the browser will interpret this structure.
- 3. Determine the impact on rendering and layout.
- 4. Suggest corrections to make the DOM valid.

ANSWER:

1. Identify missing or improperly closed elements.

The given HTML contains one improperly closed element.

Element	Status	Analysis
<html>	Properly opened and closed	Valid root element.
<head>	Properly opened and closed	Contains the page title correctly.
<title>	Properly opened and closed	Correctly defines the document title.
<body>	Properly opened and closed	Valid body section.
<div>	Properly opened and closed	Opening and closing tags are present.
<p>	Missing closing tag	The paragraph element is opened but not closed with .

Incorrect Element

The paragraph tag is missing its closing tag:

```
<p>HelloWorld
```

It should be written as:

```
<p>HelloWorld</p>
```


2. Analyze how the browser will interpret this structure.

Modern web browsers follow the HTML parsing algorithm, which is designed to handle minor HTML errors automatically.

When the browser encounters:

```
<div>
<p>HelloWorld
</div>
```

it detects that the `<div>` is closing while the `<p>` element is still open.

To maintain a valid Document Object Model (DOM), the browser automatically inserts the missing closing paragraph tag before the `</div>`.

The browser interprets the structure as:

```
<div>
  <p>HelloWorld</p>
</div>
```

Therefore, although the original HTML contains an error, the browser performs **error recovery** and constructs a valid DOM tree.

3. Determine the impact on rendering and layout.

Rendering

In this example, the webpage will display:

HelloWorld

The paragraph text appears normally because the browser automatically closes the `<p>` element.

Layout

The visual layout will generally remain correct because:

- `<div>` is a block-level element.
- `<p>` is also a block-level element.
- The browser repairs the missing closing tag before rendering.

Potential Problems

Although the page may display correctly, invalid HTML can lead to several issues:

- Different browsers may recover from errors differently.
- JavaScript DOM manipulation may not behave as expected.
- CSS selectors may produce unexpected results.
- HTML validators will report errors.
- Larger documents with multiple missing tags can produce incorrect nesting and layout problems.

Thus, relying on automatic browser correction is not considered good web development practice.

4. Suggest corrections to make the DOM valid.

The missing closing paragraph tag should be added.

Correct HTML Structure

```
<html>
<head>
  <title>TestPage</title>
</head>
<body>
  <div>
    <p>HelloWorld</p>
  </div>
</body>
</html>
```

Improvements

- Properly close every HTML element.
- Maintain consistent indentation for better readability.
- Validate the HTML using an HTML validator before deployment.
- Follow proper nesting rules to ensure a valid and predictable DOM structure.

Conclusion

The HTML document contains one structural error: the `<p>` element is not properly closed. Modern browsers automatically correct this mistake by inserting the missing `</p>` tag while constructing the DOM, allowing the page to render normally. However, depending on browser error recovery is not recommended because it may cause inconsistencies in rendering, CSS styling, and JavaScript execution in more complex documents. Adding the missing closing tag produces a valid DOM structure, improves code quality, and ensures consistent behavior across different browsers.

QUESTION 3:

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

- 1. Interpret the differences in data transmission between GET and POST.
- 2. Analyze which method is more suitable for login and why.
- 3. Identify security risks in using GET for authentication.
- 4. Suggest best practices for secure login implementation.

ANSWER:

1. Interpret the differences in data transmission between GET and POST.

GET and POST are HTTP request methods used to send data from a client to a web server, but they differ in how data is transmitted.

Feature	GET	POST
Data Transmission	Data is sent as URL query parameters.	Data is sent in the HTTP request body.
Data Visibility	Visible in the browser's address bar and URL.	Not visible in the URL; contained in the request body.
Data Length	Limited by the maximum URL length supported by the browser/server.	Can send much larger amounts of data.
Caching	Can be cached by browsers and proxies.	Generally not cached by default.
Browser History	Parameters are stored in browser history.	Request body is not stored in the URL history.
Primary Use	Retrieving resources without changing server data.	Sending sensitive or form data, such as login credentials.

Interpretation:

GET appends data to the URL, making it easily visible and suitable for retrieving information. POST sends data within the request body, making it more appropriate for transmitting sensitive information.

2. Analyze which method is more suitable for login and why.

The POST method is more suitable for login systems.

Reasons

- Usernames and passwords are sent in the **request body** rather than appearing in the URL.
- Login credentials are not stored in browser history as URL parameters.
- Sensitive information is less likely to be exposed through bookmarks or copied URLs.
- POST supports larger amounts of form data if additional authentication information is required.
- POST follows HTTP standards for operations that involve submitting confidential data.

Although POST provides better protection than GET, it does not encrypt the data by itself. Secure login requires using HTTPS (TLS/SSL) so that all transmitted data is encrypted during communication.

3. Identify security risks in using GET for authentication.

Using GET for login authentication introduces several security risks.

Security Risk	Explanation
Credentials Visible in URL	Usernames and passwords appear in the browser's address bar.
Browser History Exposure	URLs containing credentials may be saved in browser history.
Bookmarks	Users may accidentally bookmark URLs containing login information.
Server Logs	Web servers often record requested URLs, exposing credentials in log files.
Proxy and Cache Storage	Network proxies and caches may store URLs containing sensitive information.
Referrer Leakage	Credentials may be exposed through the HTTP Referer header when navigating to another website.
Shoulder Surfing	Anyone viewing the screen can easily see credentials displayed in the URL.

These risks significantly increase the possibility of unauthorized access and credential theft.

4. Suggest best practices for secure login implementation.

To implement a secure login system, the following best practices should be followed:

1. Use POST to transmit login credentials.
2. Use HTTPS (TLS/SSL) to encrypt all communication between the client and server.
3. Hash and salt passwords using secure algorithms such as bcrypt, Argon2, or PBKDF2 before storing them.
4. Never store or transmit passwords in plain text.
5. Validate all user inputs to prevent SQL Injection and Cross-Site Scripting (XSS).
6. Implement Multi-Factor Authentication (MFA) for additional security.
7. Use secure session management, including Secure and HttpOnly cookies.
8. Limit failed login attempts to reduce brute-force attacks.
9. Log authentication events without recording sensitive information such as passwords.
10. Apply proper timeout and session expiration after periods of inactivity.

Conclusion

GET and POST differ primarily in how they transmit data. GET places data in the URL, making it visible and vulnerable to exposure through browser history, server logs, bookmarks, caches, and referrer headers. POST sends data in the request body, making it the preferred method for login authentication. However, POST alone is not sufficient for complete security; it must be combined with **HTTPS**, secure password hashing, input validation, strong session management, and additional security measures such as Multi-Factor Authentication. Following these best practices ensures a secure and reliable login system.

QUESTION 4:

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

- 1. Identify missing closing tags.
- 2. Analyze issues with the tag.
- 3. Determine how browsers will handle these errors.
- 4. Suggest a corrected version of the code.

ANSWER:

1. Identify missing closing tags.

The HTML code contains several missing closing tags.

HTML Element	Status	Analysis
<!DOCTYPE html>	Valid	Correctly declares the HTML5 document type.
<html>	Properly closed	Closed with </html>.
<head>	Properly closed	Closed with </head>.
<title>	Properly closed	Closed with </title>.
<body>	Properly closed	Closed with </body>.
<h1>	Missing closing tag	Should end with </h1>.
<p>	Missing closing tag	Should end with </p>.
	Void element	Does not require a closing tag in HTML5.

Missing Closing Tags

The following tags should be added:

```
</h1>
```

```
</p>
```

2. Analyze issues with the tag.

The tag is used to display an image in an HTML document.

Current Code

```

```

Analysis

The tag is **syntactically valid in HTML5** because is a **void element**, meaning it does not require a closing tag.

However, it has one important issue:

- The **alt (alternative text)** attribute is missing.

The alt attribute is important because it:

- Improves accessibility for users who use screen readers.
- Displays alternative text if the image cannot be loaded.
- Improves search engine optimization (SEO).

A better version is:

```

```

3. Determine how browsers will handle these errors.

Modern web browsers automatically correct many HTML errors while constructing the Document Object Model (DOM).

Browser Interpretation

When the browser encounters:

```
<h1>Welcome
```

```
<p>This is a paragraph
```

it automatically inserts the missing closing tags.

The browser effectively interprets the code as:

```
<h1>Welcome</h1>
```

```
<p>This is a paragraph</p>
```

```

```

Rendering Impact

The webpage will generally display:

- Welcome as a level-1 heading.
- "This is a paragraph" as paragraph text.
- The image if image.jpg exists.

Although the page appears correctly, invalid HTML may cause:

- Differences in rendering across browsers.
- Unexpected CSS styling behavior.
- JavaScript DOM manipulation issues.
- Validation errors during development.
- Reduced accessibility because the image lacks an alt attribute.

Therefore, relying on browser error recovery is not considered good web development practice.

4. Suggest a corrected version of the code.

Correct HTML Code

```
<!DOCTYPE html>

<html>

<head>
  <title>Sample</title>
</head>

<body>
  <h1>Welcome</h1>

  <p>This is a paragraph.</p>

  
</body>
</html>
```

Improvements Made

- Added the missing </h1> closing tag.
- Added the missing </p> closing tag.
- Added the alt attribute to the tag.
- Improved indentation and formatting for readability.
- Ensured the HTML is valid and follows HTML5 standards.

Conclusion

The HTML document contains two structural errors: the <h1> and <p> elements are missing their closing tags. Although modern browsers automatically insert the missing tags during HTML parsing, relying on this behavior is not recommended because it can lead to inconsistent rendering, styling, or scripting in more complex pages. The tag is valid as a void element, but it should include an alt attribute to improve accessibility and usability. By adding the missing closing tags, including descriptive alternative text for the image, and maintaining proper formatting, the HTML becomes valid, accessible, and consistent across different browsers.

QUESTION 5:

5: Browser-Server Communication Flow

The following sequence describes a web request:

- 1. User enters a URL in the browser
- 2. DNS lookup occurs
- 3. HTTP request is sent
- 4. Server processes request
- 5. Response is returned
- 6. Browser renders page

Questions:

- 1. Interpret each step in the communication flow.
- 2. Identify where delays can occur.
- 3. Analyze the role of DNS in the process.
- 4. Suggest optimizations to improve performance.

ANSWER:

1. Interpret each step in the communication flow.

Step	Process	Interpretation
1	User enters a URL	The user types a website address (e.g., https://www.example.com) into the browser. The browser begins the process of locating and loading the requested webpage.
2	DNS Lookup	The browser contacts a Domain Name System (DNS) server to translate the domain name into its corresponding IP address, which identifies the web server on the Internet.
3	HTTP Request Sent	After obtaining the IP address, the browser establishes a connection with the server and sends an HTTP or HTTPS request asking for the requested web resource.

4	Server Processes Request	The web server receives the request, processes it, executes any required application logic or database queries, and prepares the appropriate response.
5	Response Returned	The server sends an HTTP response containing a status code (e.g., 200 OK), headers, and the requested content such as HTML, CSS, JavaScript, images, or other resources.

2. Identify where delays can occur.

Performance delays may occur at several stages of the communication process.

Step	Possible Delay	Reason
DNS Lookup	DNS resolution delay	Slow DNS servers or cache misses increase lookup time.
Network Connection	High network latency	Slow Internet connection or long geographical distance between client and server.
HTTP Request Transmission	Packet transmission delay	Network congestion or unstable connections.
Server Processing	Server-side delay	Heavy server load, inefficient code, or slow database queries.
Response Transfer	Large response size	Large HTML files, images, videos, or other resources take longer to download.
Browser Rendering	Rendering delay	Large JavaScript files, complex CSS, or extensive DOM structures increase rendering time.

3. Analyze the role of DNS in the process.

The **Domain Name System (DNS)** is a fundamental component of web communication.

Functions of DNS

- Translates human-readable domain names (e.g., `www.example.com`) into numerical IP addresses.
- Allows browsers to locate the correct web server hosting the requested website.
- Eliminates the need for users to remember numerical IP addresses.
- Enables websites to change server IP addresses without requiring users to change the website URL.

Importance of DNS

Without DNS:

- The browser cannot determine the server's IP address.
- The HTTP request cannot be sent.
- The webpage cannot be accessed.

Therefore, DNS serves as the bridge between user-friendly website names and the servers that host them.

4. Suggest optimizations to improve performance.

The following optimizations improve browser-server communication performance:

1. Use DNS caching to reduce repeated DNS lookup times.
2. Use a Content Delivery Network (CDN) to serve content from servers closer to users.
3. Enable HTTP compression (Gzip or Brotli) to reduce the size of transferred files.
4. Optimize images by compressing them and using modern formats such as WebP.
5. Minify CSS, JavaScript, and HTML to reduce file sizes.
6. Enable browser caching so frequently used resources are stored locally.
7. Use HTTPS with HTTP/2 or HTTP/3 for faster and more efficient communication.
8. Optimize server-side code and database queries to reduce processing time.
9. Reduce unnecessary HTTP requests by combining files where appropriate.
10. Use asynchronous or deferred loading for JavaScript to improve page rendering speed.

Conclusion

The browser-server communication process begins when the user enters a URL and ends when the browser renders the webpage. Each step—including DNS lookup, HTTP request transmission, server processing, response delivery, and browser rendering—is essential for successful communication. Delays can occur due to slow DNS resolution, network latency, server workload, large response sizes, or complex browser rendering tasks. DNS plays a critical role by translating domain names into IP addresses, enabling the browser to locate the correct server. Implementing optimizations such as DNS caching, CDNs, compression, browser caching, and efficient server processing significantly improves website performance, reduces loading time, and enhances the overall user experience.